Robotic Process Automation: alertas de chuva do ChuvaRota

Global Solution 2026.2 · Engenharia de Software · 4º ano

1. Integrantes

Vinicius Barros, RM 97824

2. CitizenTech e problema resolvido

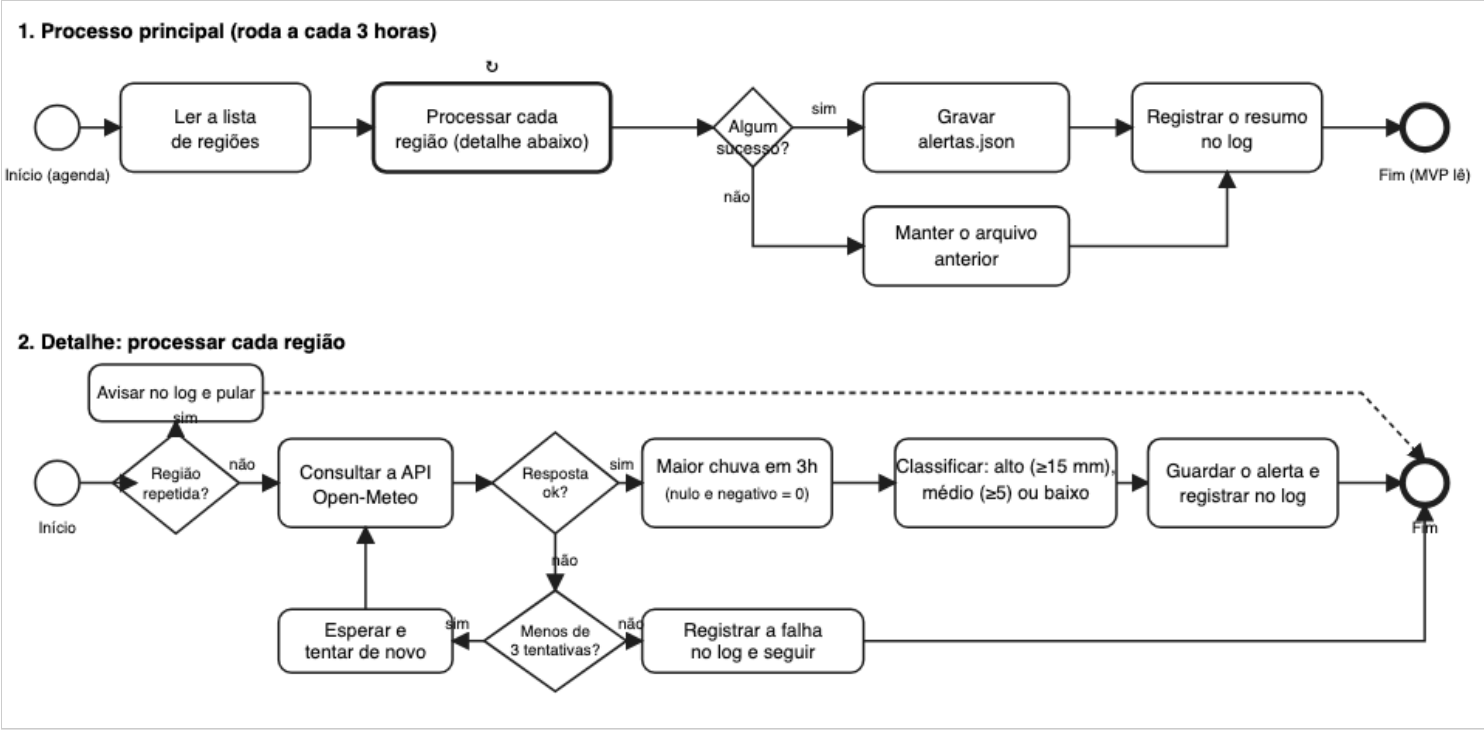
O ChuvaRota mostra o risco de chuva por região de São Paulo para quem se desloca a pé ou de ônibus. O problema operacional dos bastidores: alguém precisaria consultar a previsão de cada região e atualizar à mão o arquivo que o MVP exibe. Sem isso, o aviso fica velho e deixa de servir ao cidadão.

3. Objetivo e benefício para o cidadão

Atualizar os alertas sozinha, a cada 3 horas, sem ninguém mexer. O cidadão abre o MVP e vê o risco por região com a previsão do dia, em vez de dados desatualizados.

4. Processo e fluxo

O fluxo em BPMN mostra o processo principal e o detalhe do tratamento de cada região. O PDD completo está em `rpa/docs/PDD.md`.



Fluxo BPMN do processo.

5. Tecnologias e justificativa

Tecnologia	Por que
Python	Simples, fica versionado no GitHub e é fácil de testar.

API Open-Meteo	Pública e gratuita, não exige chave (nada de segredo no código).
GitHub Actions	Agenda a execução a cada 3 horas sem servidor próprio e grava o resultado no repositório.
pytest	Testa sucesso, decisão e falha sem depender da internet.

6. Regras de negócio, decisões e processamento

- **Repetição:** o programa passa por cada uma das 5 regiões da lista.
- **Decisão:** pega o maior acumulado de chuva em 3 horas seguidas. 15 mm ou mais é **alto**, 5 mm ou mais é **médio**, abaixo disso é **baixo**.
- **Dados inválidos:** valores nulos ou negativos contam como 0.
- **Região repetida:** é ignorada e registrada no log.

7. Exceções e recuperação

Se a API falha, o programa tenta 3 vezes (esperando 2 e 4 segundos), registra o erro e segue para a próxima região. Se nenhuma região der certo, o arquivo `alertas.json` anterior é mantido e o programa termina com erro, que o GitHub Actions marca como falha. Nada falha em silêncio.

8. Logs e auditoria

O log tem data e hora, região, chuva, risco e erros, sem nenhum dado pessoal. Execução normal:

```
--- execução normal ---
2026-11-04 22:36:41,115 INFO regiao=se mm_3h=0.0 risco=baixo
2026-11-04 22:36:41,966 INFO regiao=pinheiros mm_3h=0.2 risco=baixo
2026-11-04 22:36:42,832 INFO regiao=mooca mm_3h=0.0 risco=baixo
2026-11-04 22:36:44,244 INFO regiao=santo-amaro mm_3h=0.2 risco=baixo
2026-11-04 22:36:44,967 INFO regiao=tatuape mm_3h=0.2 risco=baixo
2026-11-04 22:36:44,967 INFO fim ok=5 falhas=0
```

Execução com a API inacessível (simulada):

```
--- API inacessível ---
2026-11-04 22:32:41,281 WARNING regiao=se tentativa=1 erro=ConnectionError
2026-11-04 22:32:41,283 WARNING regiao=se tentativa=2 erro=ConnectionError
2026-11-04 22:32:41,284 WARNING regiao=se tentativa=3 erro=ConnectionError
2026-11-04 22:32:41,284 ERROR regiao=se falhou: falhou depois de 3 tentativas
2026-11-04 22:32:41,285 WARNING regiao=pinheiros tentativa=1 erro=ConnectionError
2026-11-04 22:32:41,287 WARNING regiao=pinheiros tentativa=2 erro=ConnectionError
2026-11-04 22:32:41,288 WARNING regiao=pinheiros tentativa=3 erro=ConnectionError
2026-11-04 22:32:41,288 ERROR regiao=pinheiros falhou: falhou depois de 3 tentativas
2026-11-04 22:32:41,288 INFO fim ok=0 falhas=2
```

9. Integração com o MVP

O resultado é gravado em `mvp/data/alertas.json` e o MVP o entrega pela rota `/alertas` e na tela inicial. O workflow do GitHub Actions também foi executado de verdade na nuvem, com sucesso:
`https://github.com/ViniciusBarrosFiap/chuvarota/actions/runs/41949424767 2026-11-05T01:34:23Z`



Tela do MVP exibindo os alertas gerados pelo RPA.

10. Testes realizados

Cenário	Teste	Resultado
Sucesso	`test_sucesso`: duas regiões, risco alto gravado	passou
Decisão	`test_decisao`: limites 0, 5 e 15 mm; nulo e negativo viram 0	passou
Falha	`test_falha_e_duplicado`: uma região com API fora e uma repetida	passou
Falha total	`test_falha_total_nao_apaga_arquivo`: arquivo anterior preservado	passou

```
tests/test_rpa.py::test_sucesso PASSED                                   [ 25%]
tests/test_rpa.py::test_decisao PASSED                                   [ 50%]
tests/test_rpa.py::test_falha_e_duplicado PASSED                         [ 75%]
tests/test_rpa.py::test_falha_total_nao_apaga_arquivo PASSED             [100%]
============================== 4 passed in 0.09s ===============================
```

11. Segurança e LGPD

O programa não usa chave nem senha (a API é pública), então não há segredo no código; o repositório também roda o Gitleaks a cada push. Os dados são só previsão de chuva por região, sem nenhum dado pessoal, e o log guarda apenas região, chuva e risco.

12. Código principal

```
import json
import logging
import sys
import time
from datetime import datetime, timezone
from pathlib import Path

import requests
```

```
REGIOES = [
    {"id": "se", "nome": "Sé", "lat": -23.55, "lon": -46.63},
    {"id": "pinheiros", "nome": "Pinheiros", "lat": -23.56, "lon": -46.69},
    {"id": "mooca", "nome": "Mooca", "lat": -23.55, "lon": -46.60},
    {"id": "santo-amaro", "nome": "Santo Amaro", "lat": -23.65, "lon": -46.71},
    {"id": "tatuape", "nome": "Tatuapé", "lat": -23.54, "lon": -46.57},
]
URL = "https://api.open-meteo.com/v1/forecast"
SAIDA = Path(__file__).parents[2] / "mvp" / "data" / "alertas.json"
LOG = Path(__file__).parents[1] / "logs" / "rpa.log"

logging.basicConfig(filename=LOG, level=logging.INFO, format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger("rpa")


def classificar(mm_3h):
    if mm_3h >= 15:
        return "alto"
    if mm_3h >= 5:
        return "medio"
    return "baixo"


def max_3h(horas):
    # valor nulo ou negativo vira 0
    v = [x if isinstance(x, (int, float)) and x >= 0 else 0.0 for x in horas]
    return max(sum(v[i:i + 3]) for i in range(max(len(v) - 2, 1)))


def buscar(regiao, tentativas=3):
    for t in range(1, tentativas + 1):
        try:
            r = requests.get(URL, params={"latitude": regiao["lat"], "longitude": regiao["lon"],
                                          "hourly": "precipitation", "forecast_days": 1,
                                          "timezone": "America/Sao_Paulo"}, timeout=15)
            r.raise_for_status()
            return r.json()["hourly"]["precipitation"]
        except (requests.RequestException, KeyError, ValueError) as e:
            log.warning("regiao=%s tentativa=%d erro=%s", regiao["id"], t, type(e).__name__)
            if t < tentativas:
                time.sleep(2 * t)
    raise RuntimeError(f"falhou depois de {tentativas} tentativas")


def executar(regioes=REGIOES, saida=SAIDA):
    alertas, vistos, ok, falhas = [], set(), 0, 0
    for g in regioes:
        if g["id"] in vistos:
            log.warning("regiao=%s duplicada, ignorada", g["id"])
            continue
        vistos.add(g["id"])
        try:
            mm = max_3h(buscar(g))
        except Exception as e:
            log.error("regiao=%s falhou: %s", g["id"], e)
            falhas += 1
            continue
        risco = classificar(mm)
        alertas.append({"id": g["id"], "regiao": g["nome"], "mm_3h": round(mm, 1), "risco": risco})
        log.info("regiao=%s mm_3h=%.1f risco=%s", g["id"], mm, risco)
        ok += 1
    # só grava se veio dado novo, para não apagar o último arquivo bom
    if ok:
        saida.write_text(json.dumps({"atualizado_em": datetime.now(timezone.utc).isoformat(),
                                     "alertas": alertas}, ensure_ascii=False, indent=2), encoding="utf-8")
    log.info("fim ok=%d falhas=%d", ok, falhas)
    return {"ok": ok, "falhas": falhas}
```

```
if __name__ == "__main__":
    res = executar()
    print(res)
    sys.exit(1 if res["ok"] == 0 else 0)
```

13. Links

Repositório: https://github.com/ViniciusBarrosFiap/chuvarota (pasta `rpa/`)
Vídeo: https://github.com/ViniciusBarrosFiap/chuvarota/blob/main/rpa/assets/video_rpa.mp4